Convenience Store Data Analysis

ITEC 2600 Section B

Professor: Amanda Tian

Davide Eolio #219658723

Talal Almahayni #220305801

Mohamed Mallah #219712314

Sample Data:

	A	B	C	D	E	F	G	H	I	J	K	L
1	TransactionID	Date	Time	ItemID	ItemName	BusinessCost	Quantity	UnitCost	TotalSale	TaxTotal Sale	Category	Payment Type
2	20251201001	2025-12-01	8:05:12	C-101	Coffee (Large)	0.89	1	2.5	2.5	0.33	Beverage	Credit Card
3	20251201002	2025-12-01	8:15:30	S-205	Potato Chips	0.75	2	1.99	3.98	0.52	Snack	Cash
4	20251201003	2025-12-01	9:40:01	D-402	Daily Newspaper	0.93	1	4.5	4.5	0.59	Other	Debit Card
5	20251201004	2025-12-01	10:10:45	C-105	Soda Can	0.6	3	1.75	5.25	0.68	Beverage	Debit Card
6	20251201005	2025-12-01	11:30:22	S-202	Energy Bar	1.29	1	2.99	2.99	0.39	Snack	Cash
7	20251201006	2025-12-01	12:45:00	C-101	Coffee (Large)	0.89	1	2.5	2.5	0.33	Beverage	Debit Card
8	20251201007	2025-12-01	13:55:18	S-205	Potato Chips	0.75	1	1.99	1.99	0.26	Snack	Cash
9	20251201008	2025-12-01	14:15:30	B-303	Bread Loaf	1.75	1	4.5	4.5	0.59	Other	Credit Card
10	20251201009	2025-12-01	15:00:00	C-105	Soda Can	0.6	2	1.75	3.5	0.46	Beverage	Debit Card

TransactionID: unique codes used for the entire process of a sale
ItemID: A code unique given to the item being sold
Date & Time: represents the date and time of the sale
BusinessCost: The amount the item costs for the store
Quantity: Amount of the item sold during said transaction
UnitCost: How much the items cost for customers
TotalSale: How much the total sale comes to (Quantity * UnitCost)
TaxTotalSale: Amount of tax owed on items
Category: The category of the item, related to the ItemID
Payment type: How the customers have paid

- **Introduction:**

The goal of this study is to provide an analysis using weekly sales data from a local convenience store. The data outline we are using was provided to us by one of the group members who works at a convenience store, and we were able to simulate data to fill out the table. An examination of each store's transaction history from December 1 until December 7th, to enable a secretary to organize daily working data to generate a monthly report to find out which are the top-selling categories of products, and determine the overall profit of the week. This information can be used by the owner to improve their decision-making concerning inventory purchases and sales.

- **Part 1 Code:**

```
T = readtable("ITEC2600data.xlsx");
T.Category = categorical(T.Category);
T.PaymentType = categorical(T.PaymentType);
T.TransactionID = categorical(T.TransactionID);
T.Profit = T.TotalSale - (T.Quantity .* T.BusinessCost);
T.Revenue = T.TotalSale;
```

- **Part 1 Explanation:**

This section of the code loads the data from the Excel file named "ITEC2600data.xlsx" into a table. It takes the columns from the Excel file: Category, PaymentType, and TransactionID,

and transforms them into categorical types to allow the instantiation of the table and plotting to be able to manipulate the data.

- **Part 2 Code:**

```
fprintf("\nRevenue Summary:\n\n");
fprintf("Max revenue per sale: %.2f\n", max(T.Revenue));
fprintf("Min revenue per sale: %.2f\n", min(T.Revenue));
fprintf("Total revenue: %.2f\n", sum(T.Revenue));
fprintf("Average revenue per sale: %.2f\n", mean(T.Revenue));
fprintf("\nProfit Summary:\n\n");
fprintf("Total profit: %.2f\n", sum(T.Profit));
fprintf("Average profit per sale: %.2f\n\n", mean(T.Profit));
```

- **Part 2 Output:**

Revenue Summary:

Max revenue per sale: 7.50
Min revenue per sale: 1.75
Total revenue: 710.54
Average revenue per sale: 3.38

Profit Summary:

Total profit: 464.04
Average profit per sale: 2.21

- **Part 2 Explanation:**

This section of the code has the role of outputting the categorical data obtained from the Excel data sheet. According to the result provided by this program, the store achieved a Total Revenue of $710.54 and a total profit of $464.04 during the period that was examined. The profit margins of this convenience store are quite strong, which indicates that they have kept their costs down relatively well in comparison to their revenue. Based on an analysis of the sales transactions, on average, each sale yielded approx. $3.38 of revenue per transaction. However, the maximum revenue per transaction only hit $7.50. The relatively low revenue per transaction indicates that this business is classified as a convenience store for several reasons; primarily, because the majority of sales occur as a result of a customer purchasing a small quantity of frequently purchased snack and beverage items that are usually low-cost rather than large ticket high-priced items.

- **Part 3 Code:**

```
startDay = min(T.Date);
endDay = max(T.Date);
```

```
daysNum = days(endDay - startDay) + 1;
daysVec = startDay + caldays(0 : daysNum - 1);
dailyQuantity = zeros(daysNum,1);
dailyRevenue = zeros(daysNum,1);
dailyProfit = zeros(daysNum,1);
for i = 1:daysNum
  index = (T.Date == daysVec(i));
  dailyQuantity(i) = sum(T.Quantity(index));
  dailyRevenue(i) = sum(T.Revenue(index));
  dailyProfit(i) = sum(T.Profit(index));
end
daysVec = daysVec(:);
DailyTable = table(daysVec, dailyRevenue, dailyProfit, dailyQuantity, 'VariableNames',
{'Day','TotalRevenue','TotalProfit','TotalQuantity'});
disp(DailyTable)
```

- **Part 3 Output:**

Day	TotalRevenue	TotalProfit	TotalQuantity
01-Dec-2025	103.37	67.64	41
02-Dec-2025	94.12	61.43	36
03-Dec-2025	87.63	57.43	33
04-Dec-2025	87.63	57.43	33
05-Dec-2025	87.63	57.43	33
06-Dec-2025	133.07	86.49	55
07-Dec-2025	117.09	76.19	47

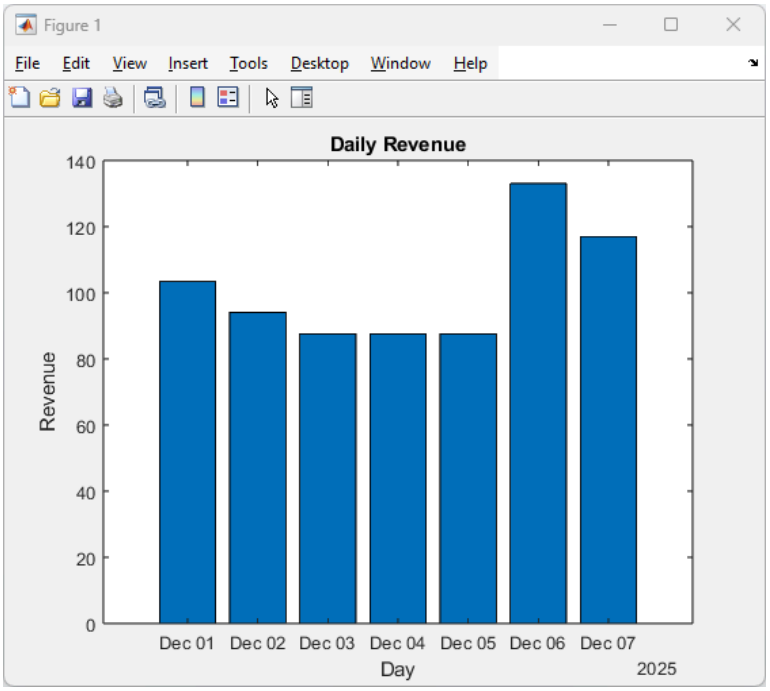
- **Part 3 Explanation:**

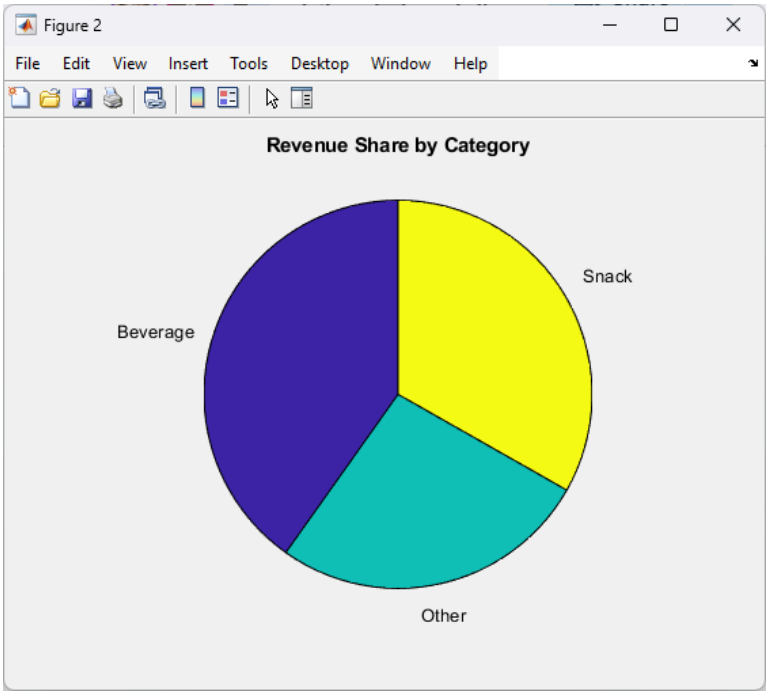
This part of the code uses the date range to create a vector that includes each date between the start and last days of transaction. A table is created, with the titles: Day, TotalRevenue, TotalProfit, TotalQuantity. The data for “Day” was taken from the daysVec vector, and the other three columns took their data from the dailyRevenue, dailyProfit, and dailyQuantity vectors. The for loop was created to gather the data for these column vectors.

- **Part 4 Code:**

```
figure
bar(DailyTable.Day, DailyTable.TotalRevenue)
title('Daily Revenue')
xlabel('Day')
ylabel('Revenue')
CategoryTable = groupsummary(T, 'Category', 'sum', 'Revenue');
```

```
figure
pie(CategoryTable.sum_Revenue, string(CategoryTable.Category))
title('Revenue Share by Category')
```

- **Part 4 Output:**
- 

- 

- **Part 4 Explanation:**

There are two important visuals in this scripted piece that will help an executive make intelligent decisions about their business. The first of these visuals is a bar chart of daily revenue, and the second visual is a pie chart that displays revenue performance for various categories. Daily revenue bar charts are created using the bar() function, and the total revenue for a business (TotalRevenue) is plotted adjacent to the days of the week (Day). This allows

the bisecting of the weekly sales trend. Using these trends, the business owner can easily assess which days will require an elevated number of staff members due to the increased staffing level as the higher volume of customers visiting on certain days. For example, the business owner was able to leverage the group summary feature to accumulate their revenues on a product-by-product basis, so that the revenue amount for each product category was placed into a CategoryTable. The CategoryTable is displayed using the pie chart that was created using the pie function. This pie chart divides the category revenue amounts by percentage of the total revenue amount for all product categories providing the business owner with a clearer understanding of which product categories generate the highest total revenue (i.e. Snacks vs. Beverages) and provides a basis for prioritizing the ordering and purchasing of those product categories to help avoid out of stock situations which would lead to a loss of potential revenues.

- **Part 5 Code:**

```
T.ItemName = string(T.ItemName);
itemList = string([]);
for i = 1:height(T)
  name = T.ItemName(i);
  exists = 0;
  for j = 1:length(itemList)
    if itemList(j) == name
      exists = 1;
      break;
    end
  end
  if ~exists
    itemList(end + 1) = name;
  end
end
totalQty = zeros(length(itemList),1);
totalProf = zeros(length(itemList),1);
for i = 1:length(itemList)
  index = strcmp(T.ItemName, itemList(i));
  totalQty(i) = sum(T.Quantity(index));
  totalProf(i) = sum(T.Profit(index));
end
ItemSummaryTable = table(itemList', totalQty, totalProf, 'VariableNames', {'ItemName',
'TotalQuantity', 'TotalProfit'});
disp(ItemSummaryTable);
```

- **Part 5 Output:**

ItemName	TotalQuantity	TotalProfit
"Coffee (Large)"	54	86.94
"Potato Chips"	51	63.24
"Daily Newspaper"	28	99.96
"Soda Can"	86	98.9
"Energy Bar"	45	76.5
"Bread Loaf"	14	38.5

- **Part 5 Explanation:**

The following part of the code analyzes the sales of each item from the data. The item names were converted to strings to avoid any issues, then the code loops through every single transaction identifying every unique item that was sold. Then for each time on the list, the program takes every relevant sale to the item and extracts the profit and total quantity sold, the program performs this execution for all items. All these items are stored in a table displaying each item with its total quantity sold and total profit made. This analysis directly helps with recognizing what items contribute the most to the store and have the highest profit margins, highly benefiting the store.

- **Part 6 Code:**

```
bestQty = totalQty(1);
bestQtyItem = itemList(1);
bestProf = totalProf(1);
bestProfItem = itemList(1);
for i = 2:length(itemList)
  if totalQty(i) > bestQty
     bestQty = totalQty(i);
     bestQtyItem = itemList(i);
  end
  if totalProf(i) > bestProf
     bestProf = totalProf(i);
     bestProfItem = itemList(i);
  end
end
fprintf("\nBEST SELLERS\n\n");
fprintf("Top by Quantity: %s (%.0f units)\n", bestQtyItem, bestQty);
fprintf("Top by Profit:   %s ($%.2f)\n", bestProfItem, bestProf);
```

- **Part 6 Output:**

BEST SELLERS

Top by Quantity: Soda Can (86 units)
Top by Profit: Daily Newspaper ($99.96)

- **Part 6 Explanation:**

The first section defines an easy linear search method for identifying the best-selling items within a store's inventory based on both quantity and profits generated from those sales. Two variables are initially assigned the best quantity and profit values, and begin with the contents of the first product on the list. Within a loop that processes through the different products, checks are made to see if the totals of the product currently reviewed exceed the previous top sellers in terms of both quantity and/or profit, using multiple conditional statements to determine whether the current item will become a better seller than previously identified products from the same list. The first section defines an easy linear search method for identifying the best-selling items within a store's inventory based on both quantity and profits generated from those sales. Two variables are initially assigned the best quantity and profit values, and begin with the contents of the first product on the list. Within a loop that processes through the different products, checks are made to see if the totals of the product currently reviewed exceed the previous top sellers in Terms of both quantity and/or profit, using multiple conditional statements to determine whether the current item will become a better seller than previously identified products from the same list.

Appendix Code:

```
T = readtable("ITEC2600data.xlsx");

T.Category = categorical(T.Category);
T.PaymentType = categorical(T.PaymentType);
T.TransactionID = categorical(T.TransactionID);

T.Profit = T.TotalSale - (T.Quantity .* T.BusinessCost);
T.Revenue = T.TotalSale;

fprintf("\nRevenue Summary:\n\n");
fprintf("Max revenue per sale: %.2f\n", max(T.Revenue));
fprintf("Min revenue per sale: %.2f\n", min(T.Revenue));
fprintf("Total revenue: %.2f\n", sum(T.Revenue));
fprintf("Average revenue per sale: %.2f\n", mean(T.Revenue));
fprintf("\nProfit Summary:\n\n");
fprintf("Total profit: %.2f\n", sum(T.Profit));
fprintf("Average profit per sale: %.2f\n\n", mean(T.Profit));

startDay = min(T.Date);
endDay = max(T.Date);
daysNum = days(endDay - startDay) + 1;
daysVec = startDay + caldays(0 : daysNum - 1);

dailyQuantity = zeros(daysNum,1);
dailyRevenue = zeros(daysNum,1);
```

```
dailyProfit = zeros(daysNum,1);

for i = 1:daysNum
  index = (T.Date == daysVec(i));
  dailyQuantity(i) = sum(T.Quantity(index));
  dailyRevenue(i) = sum(T.Revenue(index));
  dailyProfit(i) = sum(T.Profit(index));
end

daysVec = daysVec(:);
DailyTable = table(daysVec, dailyRevenue, dailyProfit, dailyQuantity,
'VariableNames', {'Day','TotalRevenue','TotalProfit','TotalQuantity'});
disp(DailyTable)

figure
bar(DailyTable.Day, DailyTable.TotalRevenue)
title('Daily Revenue')
xlabel('Day')
ylabel('Revenue')
CategoryTable = groupsummary(T, 'Category', 'sum', 'Revenue');
figure
pie(CategoryTable.sum_Revenue, string(CategoryTable.Category))
title('Revenue Share by Category')

T.ItemName = string(T.ItemName);
itemList = string([]);

for i = 1:height(T)
  name = T.ItemName(i);
  exists = 0;
  for j = 1:length(itemList)
      if itemList(j) == name
          exists = 1;
          break;
      end
  end
  if ~exists
      itemList(end + 1) = name;
  end
end

totalQty = zeros(length(itemList),1);
totalProf = zeros(length(itemList),1);

for i = 1:length(itemList)
  index = strcmp(T.ItemName, itemList(i));
  totalQty(i) = sum(T.Quantity(index));
  totalProf(i) = sum(T.Profit(index));
end

ItemSummaryTable = table(itemList', totalQty, totalProf, 'VariableNames',
{'ItemName', 'TotalQuantity', 'TotalProfit'});
disp(ItemSummaryTable);
```

```
bestQty = totalQty(1);
bestQtyItem = itemList(1);
bestProf = totalProf(1);
bestProfItem = itemList(1);

for i = 2:length(itemList)
  if totalQty(i) > bestQty
      bestQty = totalQty(i);
      bestQtyItem = itemList(i);
  end
  if totalProf(i) > bestProf
      bestProf = totalProf(i);
      bestProfItem = itemList(i);
  end
end

fprintf("\nBEST SELLERS\n\n");
fprintf("Top by Quantity: %s (%.0f units)\n", bestQtyItem, bestQty);
fprintf("Top by Profit:   %s ($%.2f)\n", bestProfItem, bestProf);
```